网络编程（仿nginx）

- 日志初始化
  - ngx_log_init();
- 信号量初始化
  - ngx_init_signals(); //
    - ngx_signal_handler（）//每个信号量对应的处理函数
      - ngx_process_get_status(),//获取子进程的结束状态，每个处理函数最后都会有
        - 信号的作用是：让 master 进程能安全、异步地响应外部命令（如退出、重载）和内部事件（如子进程崩溃），从而实现进程管理与控制。
- 初始化socket
  - g_socket.initialize()
    - freeConfig()获取配置文件，方便从配置表获取listen最大连接数之类的设置
    - ngx_open_listening_sockets()
      - 执行一些常规操作
        根据到的配置listen最大连接数来循环创建socket
        1.socket
        2.bind()绑定
        3.listen()创建两列条存进来的连接
      - 将socket设置为非阻塞模式
- 环境变量搬家
- 守护进程创建
  - fork()创建子进程
- 核心处理
  - ngx_master_process_cycle();
    - ngx_start_worker_processes(workprocess),//创建worker子进程
      - for循环创建子进程
        ngx_spawn_process(),"worker process");
        创建多少个由读取的配置文件来定
      - 所有子进程在这里循环
        ngx_worker_process_cycle(ccln,(unsignedprocname);
        - ngx_worker_process_init(int ccln)
          初始化子进程
          - 重新设置标题
            - ngx_setproctitle(pprocname), //设置标题
          - 初始化子进程
            这个线程池等广播唤醒，回收检测
            - ServerSendQueueThread
              用来发送数据的线程
              - 1.首先是防御性编程
              2.然后是全部发送完成
              3.没有发送完，注册epoll上检区完成啊下的文件
              4.如果超时就直接释放内存
              - sendproc
                拿到发送出去的字节数
              - FreeMemory()释放内存
              - ngx_epoll_oper_event()注册epoll事件
            - ServerRecyConnectionThread
              每7.1毫米回收连接的线程
              - 1.超过设定时间就回收
              2.整个程序要退出就回收
              - 具体流程是
              1.先上锁取连接时间
              2.while无限循环判断是否超时
              调用标记连接回收的函数
              - ngx_free_connection(p_Conn),//标记插槽
              重置这个插槽的信息状态，然后把地址连接池置。
            - ServerTimeQueueMonitorThread(void* threadData)
              处理到期不发心跳包的用户
              - GetOverTimeTimer
                拿到所有超时的时间点
                - 获取最早时间
                - 超时检测时
                - "procPingTimeOutChecking"
                  内存释放
            - ngx_epoll_init()
              初始化epoll程序内容
              - 1.创建连接池
              2.遍历最接字队列Listen绑定一块内存
              （1）从连接池拿一个连接，让连接对象绑定对应fd；
              3.让每一个连接对象的函数指针都绑定新连接的函数(p_Conn->rhandler = &CSocekt::ngx_event_accept;
              &ngx_epoll_oper_event)放每个epoll
              - initconnection()
                初始化连接池
              - lengx_connection_T p_Conn=ngx_get_connection(fd);//也获取空闲连接对象
              - p_Conn->rhandler = &CSocekt::ngx_event_accept;
                //开启和第3步函数中设置监听函数，因为监听端口是用来等对方连接的发送三次握手的，所以监听端口只关心的是连接事件
                - 1.用accept拿到队列的连接
                2.接着就是一系列的操作编程
                3.从连接池拿一个连接和客户端连接绑定
                4.绑定数据来时读处理函数
                5.绑定数据写处理函数
                - new(c->rhandler = &CSocekt::ngx_read_request_handler; //
                  数据读处理函数
                - 1.recv（）接收多少多少个字节
                如果收到完整的消息（包头）
                则打
                ngx_wait_request_handler_proc_p(pConn,pRood);//根据调用专门对付包头处理完整的函数去处理吧。
                - recvproc（）//整原包装了recv（）函数，知道recv少了0就关闭连接
                  - 因为=等于0说收到字节
                  pCloseSocketProc（）
                  所以调用这个函数关掉连接
                - ngx_wait_request_handler_proc_p(pConn,pRood);
                  //连接回收队列相关的互斥量释放
                  - pthread_mutex_destroy(&m_connectionMutex);
                  //连接相关互斥量释放
                  pthread_mutex_destroy(&m_sendMessageQueueMutex);　//发消息互斥量释放
                  pthread_mutex_destroy(&m_recyconnqueueMutex);　//连接回收队列相关的互斥量释放
                  pthread_mutex_destroy(&m_timequeueMutex);
                  //时间检测队列相关的互斥量释放
                  sem_destroy(&m_semEventSendQueue);
                  //发消息相关线程信号量释放
                - 1.判断能否顺意攻击不是就让入消息队列并触发线程处理消息
                g_threadpool.inMsgRecvQueueAndSignal(pConn->precMemPointer);
                  - 最后(void *)线程干活
                - 1.如果没发送完调用 sendproc()获取成功发送的字节数
                2.如果发送完就调用注销epoll事件
                3.p_memory->FreeMemory(pConn->psendMemPointer);释放内存
            - 1.初始化连接池
            2.遍历listen绑定的socket
            3.根据listen的数量创建线程
            - g_socket.threadRecvProcFunc(pMsbuf);　//
              初期遍历线程的函数
              - 1.记录消息头和头和包体的位置
              2.用crc32判断是否合法
              3.根据msgcode来实现
              - 如果值为0，则是反馈注册功能
                - void CSocekt::msgSend(char *psendBuf);
                //一个将发送的消息放到发送队列里
              - 如果值为4，则是实现登录功能
                - void CSocekt::msgSend(char *psendBuf);
                //一个将发送的消息放到发送队列里
            - ThreadFunc（）
              每一个线程创建时都会执行这个函数
              1.获取自身id
              2.进入while死循环
              3.pthread_cond_wait()所有线程都会阻塞在这里
              不会执行下一步代码，等待信号唤醒
              4.拿出第一个元素
              5.处理消息
              - g_memory->FreeMemory(pMsbuf);//释放消息内存
            - 1.初始化socket
            2.初始化线程
            3.初始化时间
            4.初始化内存
            5.初始化信号量
            6.初始化信号量
            7.初始化信号量
            8.初始化信号量
            9.初始化信号量
            10.初始化信号量
            11.初始化信号量
            12.初始化信号量
            13.初始化信号量
            14.初始化信号量
            15.初始化信号量
            16.初始化信号量
            17.初始化信号量
            18.初始化信号量
            19.初始化信号量
            20.初始化信号量
            21.初始化信号量
            22.初始化信号量
            23.初始化信号量
            24.初始化信号量
            25.初始化信号量
            26.初始化信号量
            27.初始化信号量
            28.初始化信号量
            29.初始化信号量
            30.初始化信号量
            31.初始化信号量
            32.初始化信号量
            33.初始化信号量
            34.初始化信号量
            35.初始化信号量
            36.初始化信号量
            37.初始化信号量
            38.初始化信号量
            39.初始化信号量
            40.初始化信号量
            41.初始化信号量
            42.初始化信号量
            43.初始化信号量
            44.初始化信号量